Огляд дисципліни

Тема 1. Вступ до NoSQL

Наукова публікація 1

Тема 2. Основи MongoDB та CRUD

Тема 3. Експлуатація та адміністрування MongoDB

Завдання 1. Аналітична платформа для музичного стрімінгового сервісу

Тема 4. Wide-Column бази даних — HBase

Тема 5. Бази даних типу ключ-значення

Тема 6. Вступ до векторних баз даних

Наукова публікація 2

Тема 7. Векторні бази даних продовження

Завдання 2. Семантичний пошук за науковими статтями

👓 Навчання

Завдання 2. Семантичний пошук за науковими статтями

💻 **Домашнє завдання**

Тема 8. Вступ по графових баз даних

Тема 9. Графові бази даних: продовження

Завдання 3. Граф знань для рекомендаційної системи

Тема 10. Бази даних для зберігання часових рядів

Фінальне тестування 1.0

Фінальне тестування 2.0

🔹 Налаштування оточення

Pinecone

Зареєструйтеся на pinecone.io — безкоштовний тір (Starter) підтримує один індекс і до 100 000 векторів, чого вистачає для даного завдання.

Після реєстрації:

1. Створіть API-ключ у розділі **API Keys**.
2. Збережіть його у файл `.env` у корені проєкту (файл не комітити — додайте в `.gitignore`):

```
PINECONE_API_KEY=your-api-key-here
```

Python-залежності

Створіть файл `requirements.txt`:

```
pinecone-client==4.1.0
sentence-transformers==3.0.1
rank-bm25==0.2.2
kaggle==1.6.14
pandas==3.0.3
numpy==2.4.6
tqdm==4.66.4
pyarrow==23.0.1
python-dotenv==1.0.1
```

Встановлення:

```
pip install -r requirements.txt
```

🔹 Частина 1 — Підготовка даних і вибір інструментів

Мета: завантажити датасет, вибрати і отримати ембеддинги.

1.1. Завантаження і підготовка датасету

Завантажте датасет через Kaggle (знадобиться `~/.kaggle/kaggle.json`):

```
kaggle datasets download -d Cornell-University/arxiv
unzip arxiv.zip
# Результат: файл arxiv-metadata-oai-snapshot.json
# Формат: JSONL — кожен рядок є самостійним JSON-об'єктом
```

Приклад одного рядка з файлу (скорочено):

```
{
  "id": "0704.0001",
  "authors": "C. Bal\\\\'azs, E. L. Berger, P. M. Nadolsky, C.-P.
Yuan",
  "title": "Calculation of prompt diphoton production cross sections
at Tevatron and\\n  LHC energies",
  "categories": "hep-ph",
  "abstract": "  A fully differential calculation in perturbative
quantum chromodynamics...",
  "versions": [{"version": "v1", "created": "Mon, 2 Apr 2007 19:18:42
GMT"}],
  "update_date": "2008-11-26",
  "authors_parsed": [["Baláz", "C.", ""], ["Berger", "E. L.", ""]]
}
```

Виконайте скрипт.

```
# scripts/01_prepare_data.py
import json
import os
import pandas as pd
from tqdm import tqdm

INPUT_FILE  = "arxiv-metadata-oai-snapshot.json"
OUTPUT_FILE = "data/arxiv_subset.parquet"
MAX_RECORDS = 10_000

os.makedirs("data", exist_ok=True)

def extract_year(paper: dict) -> int:
    """
    Беремо рік із першої версії статті — це дата публікації на arXiv.
    update_date — дата останнього оновлення, вона може бути на роки
пізніше.
    Формат created: "Mon, 2 Apr 2007 19:18:42 GMT"
```

```
    """
    try:
        versions = paper.get("versions", [])
        if versions:
            created = versions[0]["created"]  # "Mon, 2 Apr 2007
19:18:42 GMT"
            # Рік стоїть на 4-й позиції після split по пробілу
            return int(created.split()[3])
    except (IndexError, ValueError, KeyError):
        pass
    # Запасний варіант: update_date у форматі "YYYY-MM-DD"
    return int(paper.get("update_date", "2000-01-01")[:4])

def format_authors(paper: dict) -> str:
    """
    authors_parsed — структурований список [["Прізвище", "Ініціали",
""]].
    Збираємо у читабельний рядок "Прізвище І., Прізвище І."
    Якщо authors_parsed відсутній — беремо сирий рядок authors.
    """
    parsed = paper.get("authors_parsed", [])
    if parsed:
        parts = []
        for entry in parsed[:10]:  # не більше 10 авторів
            last  = entry[0].strip() if len(entry) > 0 else ""
            first = entry[1].strip() if len(entry) > 1 else ""
            if last:
                parts.append(f"{last}{first}".strip())
        return ", ".join(parts)
    # Запасний варіант: сирий рядок авторів
    return paper.get("authors", "").replace("\\n", " ")

records = []
with open(INPUT_FILE, "r", encoding="utf-8") as f:
    for line in tqdm(f, desc="Читаємо датасет"):
        if len(records) >= MAX_RECORDS:
            break
        line = line.strip()
        if not line:
            continue
        paper = json.loads(line)

        abstract = paper.get("abstract", "").strip()
        title    = paper.get("title", "").strip()
```

```
        # Пропускаємо записи без анотації або заголовка
        if not abstract or not title:
            continue

        # categories може містити кілька категорій через пробіл:
"cs.LG cs.AI"
        # Беремо першу як основну
        categories_raw = paper.get("categories", "unknown")
        primary_category = categories_raw.split()[0]

        records.append({
            "id":       paper["id"],
            "title":    title.replace("\\n", " ").strip(),
            "abstract": abstract.replace("\\n", " ").strip(),
            "authors":  format_authors(paper),
            "year":     extract_year(paper),
            "category": primary_category,
        })

df = pd.DataFrame(records)
print(f"\\nЗавантажено статей:{len(df)}")
print(f"\\nРозподіл за категоріями (топ-10):")
print(df["category"].value_counts().head(10))
print(f"\\nРозподіл за роками:")
print(df["year"].value_counts().sort_index().tail(10))
print(f"\\nПриклад запису:")
print(df.iloc[0].to_dict())

df.to_parquet(OUTPUT_FILE, index=False)
print(f"\\nЗбережено в{OUTPUT_FILE}")
```

1.2. Вибір інструментів

У цьому завданні використовується **Pinecone** як векторна база даних і `allenai/specter2_base` як модель ембеддингів.

У README письмово дайте відповідь на такі запитання (не менше абзацу на кожне):

1. Чим Pinecone відрізняється від Qdrant і Chroma за моделлю розгортання, ліцензією і продуктивністю? У якому сценарії ви б обрали кожен із них?

2. Чому для задачі пошуку по науковим текстам обрана модель `specter2_base`, а не універсальна `all-MiniLM-L6-v2`? Знайдіть картку моделі на HuggingFace і процитуйте, для яких задач вона навчена.

3. Що написано у картці моделі про рекомендовану метрику схожості? Чому це важливо при створенні індексу?

1.3. Отримання ембеддингів

Вам необхідно реалізувати скрипт `02_embed.py` для перетворення текстових даних у векторні представлення (ембеддинги) з використанням попередньо навченої моделі з HuggingFace.

Скрипт повинен виконувати такі кроки:

1. Завантажити датасет із файлу `data/arxiv_subset.parquet` з використанням бібліотеки `pandas`.

2. Підготувати тексти для кодування:

   - для кожного запису об'єднати поля `title` і `abstract` в один рядок у форматі: `title + " [SEP] " + abstract`
   - важливо: токен `[SEP]` обов'язковий, оскільки модель навчена працювати саме з таким форматом вхідних даних.

3. Згенерувати ембеддинги текстів за допомогою моделі `allenai/specter2_base` з бібліотеки `sentence-transformers`.

4. Закодувати всі тексти в ембеддинги з урахуванням таких вимог:

   - використовувати батчеву обробку (наприклад, `batch_size=64`);
   - увімкнути відображення прогресу;
   - нормалізувати ембеддинги (`normalize_embeddings=True`).

5. Вивести в консоль:

   - загальну кількість оброблених текстів;
   - розмірність ембеддингів (очікується 768);
   - норму першого ембеддингу (повинна бути близька до 1.0).

6. Зберегти отримані ембеддинги у файл `embeddings/embeddings.npy` у форматі NumPy.

7. Перед збереженням переконатися, що директорія `embeddings` існує; за потреби створити її.

```
# scripts/02_embed.py
!!! МІСЦЕ ДЛЯ ВАШОГО КОДУ !!!
```

У файлі README надайте відповідь на наступне запитання:

Поясніть, чому при використанні нормалізованих ембеддингів (одиничної довжини) косинусна схожість (`cosine similarity`) еквівалентна скалярному добутку (`dot product`)?

🔹 Частина 2 — Завантаження даних і метадані

Спочатку створіть індекс у Pinecone, а потім напишіть скрипт `03_load_to_pinecone.py`, який завантажує підготовлені ембеддинги наукових статей у цей індекс.

Скрипт повинен виконувати такі кроки:

1. Створити індекс `arxiv-papers` у Pinecone, якщо він ще не існує, і підключитися до нього.

2. Завантажити дані:

- прочитати датасет із файлу `data/arxiv_subset.parquet`;
- завантажити ембеддинги з файлу `embeddings/embeddings.npy`.

3. Підготувати дані для завантаження:

- обробляти записи батчами (наприклад, по 200 елементів);
- для кожного запису сформувати об'єкт із:
- унікальним `id` вигляду `"paper_<номер>"`;
- ембеддингами;
- метаданими: `arxiv_id`, `title`, `abstract` (до 500 символів), `authors` (до 200 символів), `year`, `category`.

4. Завантажити дані в Pinecone батчами і показувати прогрес.

5. Після завершення завантаження вивести в консоль загальну кількість векторів в індексі.

```python
# scripts/03_load_to_pinecone.py
import os
import numpy as np
import pandas as pd
from tqdm import tqdm
from dotenv import load_dotenv
from pinecone import Pinecone, ServerlessSpec

load_dotenv()

INPUT_PARQUET = "data/arxiv_subset.parquet"
INPUT_EMBEDDINGS = "embeddings/embeddings.npy"
INDEX_NAME = "arxiv-papers"
VECTOR_DIM = 768
BATCH_SIZE = 200   # Pinecone рекомендує батчі до 200 векторів

# Ініціалізація клієнта
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
```

```
# Створюємо індекс (якщо не існує)
!!! МІСЦЕ ДЛЯ ВАШОГО КОДУ !!!
```

Чому `abstract` обрізається до 500 символів?

Pinecone обмежує сумарний розмір метаданих одного вектора до 40 KB. Повний текст анотації потрібно зберігати окремо (у parquet-файлі) і підтягувати за ID після пошуку.

◆ Частина 3 — Пошукові запити

Напишіть скрипт `04_search.py`, який виконує семантичний пошук за індексом Pinecone і порівнює результати з локальними обчисленнями.

Скрипт повинен виконувати такі кроки:

1. Підключитися до індексу `arxiv-papers` у Pinecone і завантажити модель `allenai/specter2_base`.

2. Реалізувати функцію кодування запиту в ембеддинг.

3. Виконати **чистий семантичний пошук**:

- задати запит (наприклад: `"teaching machines to recognize objects in pictures"`);

- отримати топ-5 найбільш релевантних статей;

- вивести результати з назвою, категорією, роком і частиною абстракту.

4. Виконати **пошук з фільтрацією**:

- приклад A: статті по `reinforcement learning` за останні 5 років і категорія `cs.LG`;

- приклад B: більш старі статті (до 2015 року), будь-яка категорія;

- порівняти видачу і пояснити відмінності.

5. Порівняти різні метрики схожості на локальних ембеддингах:

- завантажити всі ембеддинги з `embeddings/embeddings.npy`;

- для заданого запиту обчислити:

  - cosine similarity;
  - dot product;
  - L2-distance;

- вивести топ-5 статей для кожної метрики і порівняти результати.

У файлі README надайте відповіді на обов'язкові теоретичні запитання:

1. Чи збігаються топ-5 для cosine і dot product і чому?
2. Чи відрізняються результати для L2 і чому?
3. Що сталося б, якби ембеддинги не були нормалізовані?

```
# scripts/04_search.py
import os
import numpy as np
import pandas as pd
from dotenv import load_dotenv
from pinecone import Pinecone
from sentence_transformers import SentenceTransformer

load_dotenv()

INDEX_NAME = "arxiv-papers"
MODEL_NAME = "allenai/specter2_base"
TOP_K = 5

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
index = pc.Index(INDEX_NAME)
model = SentenceTransformer(MODEL_NAME)
df = pd.read_parquet("data/arxiv_subset.parquet")  # для отримання
повного abstract

!!! МІСЦЕ ДЛЯ ВАШОГО КОДУ !!!
```

🔹 Частина 4 — Chunking

Хоча анотації статей відносно короткі, деякі з них перевищують 512 токенів — максимум більшості sentence-transformer моделей. Крім того, chunking — базова навичка для роботи з реальними документами (PDF, книги, звіти), де тексти значно довші.

Напишіть скрипт `05_chunking.py` , який розбиває довгі анотації статей на чанки, створює ембеддинги для кожного чанка і завантажує їх в окремі індекси Pinecone для пошуку по частинах тексту.

Скрипт повинен виконувати такі кроки:

1. Вибрати 30 статей із найдовшими анотаціями.

2. Розбити тексти на чанки двома стратегіями:

- **Fixed-size chunking**: фіксована кількість слів з невеликим перекриттям між чанками;

- **Semantic chunking**: об’єднання речень до досягнення максимальної кількості слів, щоб зберегти зміст.

3. Створити окремі індекси в Pinecone для кожного типу чанків (`arxiv-chunks-fixed` і `arxiv-chunks-semantic`).

4. Для кожного чанка:

   - створити ембеддинг за допомогою моделі `allenai/specter2_base` ;
   - сформувати об’єкт з унікальним `id` , ембеддингом і метаданими: `arxiv_id` , `title` , текст чанка, номер чанка, рік, категорія.

5. Завантажувати чанки в Pinecone батчами і відображати прогрес.

6. Реалізувати функцію пошуку по чанках:

   - виконати пошук за кількома тестовими запитами;
   - вивести топ-5 результатів для кожного типу чанків з назвою статті і частиною тексту чанка.

У файлі README надайте відповіді на обов’язкові теоретичні запитання:

1. Яка стратегія дає більш осмислені чанки?
2. Чи є випадки розрізаних речень і як це впливає на ембеддинги?
3. Як розмір overlap впливає на кількість чанків і покриття тексту?

```
# scripts/05_chunking.py
import os
import re
import numpy as np
import pandas as pd
from tqdm import tqdm
from dotenv import load_dotenv
from pinecone import Pinecone, ServerlessSpec
from sentence_transformers import SentenceTransformer

load_dotenv()

MODEL_NAME = "allenai/specter2_base"
VECTOR_DIM = 768

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
model = SentenceTransformer(MODEL_NAME)
df = pd.read_parquet("data/arxiv_subset.parquet")

!!! МІСЦЕ ДЛЯ ВАШОГО КОДУ !!!
```

🔹 Частина 5 — Гібридний пошук

BM25 добре працює на точних термінах, абревіатурах і іменах авторів. Векторний пошук добре працює з перефразуваннями й синонімами. Гібридний пошук бере найкраще від обох.

Напишіть скрипт `06_hybrid_search.py` , який виконує гібридний пошук по корпусу статей, об'єднуючи BM25 і векторне ранжування за допомогою Reciprocal Rank Fusion (RRF).

Скрипт повинен виконувати такі кроки:

1. Побудувати локальний BM25-індекс за заголовками і анотаціями всіх статей.

2. Підключитися до Pinecone і використовувати модель `allenai/specter2_base` для векторного пошуку.

3. Реалізувати **Reciprocal Rank Fusion (RRF)** для об'єднання ранжованих списків BM25 і векторного пошуку:

   - об'єднувати результати двох методів;
   - формувати загальний топ-K документів.

4. Реалізувати функції пошуку:

   - BM25;
   - векторний (Pinecone);
   - гібридний (BM25 + векторний через RRF).

5. Для демонстрації виконати три запити:

   - точний термін (`"BERT fine-tuning"`);
   - ім'я автора (`"Yann LeCun convolutional networks"`);
   - перефразування без явних термінів (`"making computers understand human emotions from text"`).

6. Вивести результати для кожного методу і порівняти:

   - топ-5 BM25;
   - топ-5 векторного пошуку;
   - топ-5 гібридного пошуку з RRF, включаючи RRF-скор.

У файлі README надайте відповіді на обов'язкові теоретичні запитання:

1. Який метод дав кращий результат і чому?
2. Чи є документи в топ-5 гібридного пошуку, яких немає в топ-5 окремих методів? Чому?
3. Як зміна параметра k в RRF впливає на видачу (наприклад, k=60 vs k=1)?

```
# scripts/06_hybrid_search.py
import os
import math
import numpy as np
import pandas as pd
from dotenv import load_dotenv
from pinecone import Pinecone
from sentence_transformers import SentenceTransformer
from rank_bm25 import BM25Okapi

load_dotenv()

INDEX_NAME = "arxiv-papers"
MODEL_NAME = "allenai/specter2_base"
TOP_K = 10   # беремо ширше, щоб RRF міг переранжувати

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
index = pc.Index(INDEX_NAME)
model = SentenceTransformer(MODEL_NAME)
df =
pd.read_parquet("data/arxiv_subset.parquet").reset_index(drop=True)

!!! МІСЦЕ ДЛЯ ВАШОГО КОДУ !!!
```

◆ Частина 6 — Аналіз і висновки

Напишіть у README розгорнуті відповіді (не менше абзацу на кожен пункт):

1. **Семантичний пошук vs BM25.** Наведіть конкретні приклади запитів із вашої роботи, де кожен метод виграв. Сформулюйте загальне правило: для яких типів запитів варто надати перевагу кожному з них?

2. **Вплив розміру чанка.** Що відбувається з якістю пошуку, якщо чанк занадто маленький (10–15 слів)? Якщо занадто великий (500+ слів)? Чи є оптимальний розмір або він залежить від задачі?

3. **Невідповідна метрика.** Що сталося б, якби ми створили індекс Pinecone з метрикою `euclidean` (L2), але використовували модель, яка повертає нормалізовані вектори? Обґрунтуйте відповідь математично: виведіть зв'язок між L2 і cosine для одиничних векторів.

4. **Обмеження Pinecone Starter.** З якими обмеженнями безкоштовного тіру ви зіткнулися (або могли б зіткнутися)? Як би ви вирішили задачу, якби датасет був не 10000, а 10 мільйонів статей?

Вимоги до README

- Письмові відповіді на всі запитання з частин 1, 3, 4, 5, 6.

- Вивід кожного скрипту (скріншот терміналу або вставлений текст).

- Порівняльна таблиця методів пошуку з частини 5: три запити × три методи.

- `.env` і папки `data/`, `embeddings/` додані в `.gitignore`.

Підготовка та завантаження завдання

1. Після виконання всіх етапів, скопіюйте посилання на ваш Git-репозиторій `[Ваше прізвище]_nosql_2` і прикріпіть його в LMS.

2. Збережіть архів з усіма елементами вашої роботи і відповідями на питання у файлі README на свій комп'ютер. Збережений архів має також називатись `[Ваше прізвище]_nosql_2`.

3. Прикріпіть його в LMS у форматі `zip.`

4. Відправте завдання на перевірку.

Формат здачі

- Прикріплений архів із назвою `[Ваше прізвище]_nosql_2`.

- Посилання на Git-репозиторій з кодом, всіма елементами вашої роботи і відповідями на питання (мають бути додані в `README.md`).

Формат оцінювання

Оцінка 0-100

Критерії оцінювання

> **Критерії прийняття завдання є обов'язковою умовою оцінювання цього завдання ментором. Якщо якийсь із критеріїв не виконано, ментор відправляє завдання на доопрацювання без оцінювання.** Якщо вам «тільки уточнити»😊 або ви застопорилися на якомусь з етапів виконання — звертайтеся до ментора у Slack.

Частина	Що перевіряється	Вага
1	Дані завантажені, обґрунтування вибору моделі та БД аргументовано	10%

Частина	Що перевіряється	Вага
2	Індекс створено коректно, метадані повні, батчеве завантаження працює	15%
3	Всі три типи пошуку реалізовані, результати прокоментовані	25%
4	Обидві стратегії chunking реалізовані, порівняння осмислене	20%
5	Гібридний пошук працює, RRF реалізовано коректно, три приклади показові	20%
6	Розгорнутий аналіз компромісів, математичне обґрунтування за метриками	10%

Частина	Що перевіряється	Вага
1	Дані завантажені, обґрунтування вибору моделі та БД аргументовано	10%
2	Індекс створено коректно, метадані повні, батчеве завантаження працює	15%
3	Всі три типи пошуку реалізовані, результати прокоментовані	25%
4	Обидві стратегії chunking реалізовані, порівняння осмислене	20%
5	Гібридний пошук працює, RRF реалізовано коректно, три приклади показові	20%
6	Розгорнутий аналіз компромісів, математичне обґрунтування за метриками	10%

Скрипт, який падає з помилкою при запуску, не перевіряється. Переконайтеся, що всі скрипти запускаються послідовно від `01_` до `06_` у чистому оточенні.

⚠️ **УВАГА!!** У вас є можливість обрати підхід до виконання та можливого доопрацювання завдання:

1. задовольнитися першою отриманою оцінкою (звісно ж, якщо вона вища за прохідний бал),

2. намагатися отримати вищий бал шляхом можливого подальшого доопрацювання роботи відповідно до фідбеку ментора.

Обраний підхід до виконання ДЗ необхідно зазначити в полі для здачі **до прикріпленого завдання. За відсутності коментаря ментор дотримується першого підходу й виставляє отриману оцінку.**

Відправляйте завдання на перевірку, коли зроблено все можливе, адже кількість спроб здачі завдання впливає на отриманий бал! **За кожну наступну спробу після другої (тобто з третьої) максимально можлива кількість балів зменшується на 5.** Таким чином: 1 спроба: максимальна кількість балів — 100, 2 спроба: максимальна кількість балів — 100, 3 спроба: максимальна кількість балів — 95, 4 спроба: максимальна кількість балів — 90, 5 спроба: максимальна кількість балів — 85 і т. д. **Критерії оцінювання робіт у магістратурі GoIt Neoversity**

Від робіт очікується висвітлення вашої власної думки, інсайтів, ідей та висновків. Повністю згенеровані тексти та інші роботи за допомогою ШІ без вашого особистого внеску будуть надіслані на доопрацювання.
Пам'ятайте — ШІ дозволено використовувати лише як допоміжний інструмент у процесі роботи.

🕐 Завдання очікує виконання

🔥 Дедлайн 23:45, 5 лип.

Здай домашнє завдання!

Завантаж сюди посилання на домашнє завдання

Завантаж файл до 150 Мб

Відправити домашнє завдання